

TP n°3 : Processus et scripts BASH

Administration Système - ESIPE INFO1

Objectifs du TP : Connaître les commandes de la gestion des processus, et maîtriser les éléments de base du langage de script BASH.

1 Gestion des processus

1. Lancez votre machine virtuelle sous Linux
2. Avec l'éditeur « vim » ou « nano », saisir ce programme, nommé « pg_erreur », dans le répertoire courant.

```
#!/bin/bash
while (true)
do
    echo Bonjour
done
```

3. Enregistrez-le.
4. Rendez-le fichier « pg_erreur » exécutable.
5. Exécutez-le.
6. Que se passe-t-il ?
Boucle infini, affichant « Bonjour »
7. Arrêtez le programme par Ctrl-C.
8. Relancez le programme mais en faisant suivre son nom par un « & ».
9. Arrêtez le programme par Ctrl-C. Que constatez-vous ?
Il ne veut pas s'arrêter avec un simple Ctrl-C.
10. Pourquoi ?
Car il tourne en arrière-plan.
11. Allez sur TTY2 et connectez-vous (Ctrl Droit + F2)
12. La commande « ps » liste tous les processus actifs de la session utilisateur. Quelle commande doit-t-on taper pour voir tous les processus en cours ?
ps -ef
13. Recherchez le script que vous venez d'écrire. Comment le reconnaissez-vous ?
Grace a son PID.
14. Quel est son identifiant ?
3907209
15. Quel est l'identifiant de son père ?
3780710 (bash)
16. Lancer la commande « top ». Où est situé le programme « pg-erreur » ?
Il est situé en haut de la liste.
17. Quel est le processus qui consomme le plus de temps processeur ?
Le processeur «pg_erreur » (notre programme) consomme le plus de temps a notre processeur.

18. Quittez « top » en appuyant sur « q ».
19. Tuez le processus de « pg-erreur » à l'aide de la commande « kill » (ou avec « top »).
20. Vérifiez avec « top » que tout est rentré dans l'ordre. Comment le voyez-vous ?

Le processus «pg_erreur » a bien reçu le signal SIGTERM, on ne le voit donc plus.

21. Relancez à nouveau le script précédent, et interrompez-le avec la commande Ctrl-Z, que se passe-t'il ?

Le processus est stopé. Il est mis en pause

22. Tapez la commande « fg », que se passe-t'il ?

fg sert à reprendre la main au processus mis en arrière plan. Le processus va donc reprendre.

23. Tapez à nouveau Ctrl-Z, puis tapez la commande « bg », que se passe-t'il ? Et comment se sortir de cette situation ?

Il faudrait dans ce cas, trouver le pid du processus mis en arrière plan avec la commande « ps »
Pour par la suite lui envoyer le signal SIGTERM ou SIGKILL.

24. Expliquer brièvement le fonctionnement des commandes « nice » et « nohup »

nice sert à ajuster la priorité d'exécution d'un programme.

nohup est utilisée pour exécuter un programme ou un script en ignorant les signaux de sortie.

2 Quelques scripts simples

1. Écrivez et testez les exercices du cours sur Bash (**Exercices 1 jusqu'à 5**).
2. Écrivez un script qui demande à l'utilisateur de saisir un nombre de bits et qui affiche un message d'erreur si cette valeur est inférieure à 0, ou alors la conversion de cette valeur en nombre d'octets.
3. Reprenez le point précédent, et faites en sorte que le programme se répète tant que l'utilisateur n'a pas saisie la touche q. A la fin de l'exécution le script doit afficher la somme totale des valeurs saisies en nombre d'octets.
4. Reprenez le point précédent, et faites en sorte que la valeur saisie puisse être donnée en paramètre au programme, ou bien être saisie si elle n'a pas été trouvée en paramètre.
5. Ecrivez un script nommé *deuxfois* qui affiche le message "Entrez un mot : ", lit le mot saisi par l'utilisateur puis affiche ce mot deux fois sur la même ligne.

```
Exemple: $ ./deuxfois
Entrez un mot : toto
toto toto
$
```

6. Ecrire un script nommé *untrois* qui demande à l'utilisateur de saisir une suite de mots constituée d'au moins trois mots et qui affiche sur la même ligne le premier et le troisième mot saisis.

```
Exemple : $ ./untrois
Entrez une suite de mots : un petit coucou de Pornichet
un coucou
$
```

7. Écrire les expressions génériques qui correspondent aux motifs suivants :
 1. Les mots qui commencent par un caractère quelconque, et qui se terminent par un h

`^.+(h)$`

`^` : Indique le début de la ligne.

`+` : Correspond à un ou plusieurs caractères quelconques.

`(h)` : Capture la lettre "h" à la fin.

`$` : Indique la fin de la ligne.

2. Les mots qui contiennent la chaîne ftp mais qui ne commencent pas par un f

`^(?!f).*ftp.*`

`^` : Indique le début de la ligne.

`(?!f)` : Négation de l'assertion positive (ne doit pas commencer par "f").

`.*` : Correspond à zéro ou plusieurs caractères quelconques.

`ftp` : La chaîne "ftp".

3. Les mots qui contiennent deux fois au moins une des lettres parmi A, B ou C

`(.*[ABC]){2,}`

`(.*[ABC])` : Correspond à zéro ou plusieurs caractères quelconques suivis d'une lettre parmi A, B ou C.

`{2,}` : Indique deux occurrences ou plus.

4. Les mots de 4 caractères qui commencent par une lettre majuscule.

`^[A-Z].{3}$`

`^` : Indique le début de la ligne.

`[A-Z]` : Correspond à une lettre majuscule.

`.{3}` : Correspond à trois caractères quelconques.

`$` : Indique la fin de la ligne.

8. *Rappel*: Les variables `$1`, `$2`, `$3`... désignent les valeurs des arguments passés à un script.

En utilisant une structure *case*, écrire un script nommé *3car* qui prend en argument une chaîne de caractères et qui affiche OK si la chaîne est composée de 3 caractères seulement. Si il y en a plus il affiche un message d'erreur, sinon il n'affiche rien.

3 Quelques scripts plus "intéressants" !

Exercice 1

Avec un éditeur de votre choix, créez le script `decompte` suivant :

```
while test "$1"; do
  echo "Le paramètre 1 vaut $1"
  shift
done
```

Essayez alors de lancer `./decompte un deux trois quatre` après avoir fait un `chmod` approprié sur le fichier. Quel est l'effet de `shift` ?

décale les paramètres vers la gauche, donc maintenant

Modifier à présent le script, qui recevra au moins deux paramètres :

Le premier sera le nom d'un fichier de sortie
Le deuxième et les suivants seront des répertoires

Votre script devra ajouter une ligne de la forme « Il y a `n` fichiers dans le répertoire `machin` ». De plus, s'il rencontre des erreurs (parce que le répertoire n'existe pas ou n'est pas lisible), elles ne devront jamais apparaître à l'écran ; et en lançant votre script deux fois 3 de suite, le fichier de sortie ne devra contenir que les informations relatives à la dernière exécution.

Exemple d'utilisation :

```
$ ./decompte /tmp/sortie /etc /usr
$ cat /tmp/sortie
Il y a 296 fichiers dans /etc
Il y a 12 fichiers dans /usr
$ cat /tmp/sortie
Il y a 6 fichiers dans /home
$
```

Exercice 2

La commande `test x -le y` permet de tester si l'entier `x` est plus petit ou égal à `y`. Et la commande `test x = y` teste si les chaînes de caractères `x` et `y` sont identiques. Sachant cela, écrivez un script `sequence.sh` `[-c t]` `debut fin` qui écrit tous les nombres entiers entre `debut` et `fin` inclus, un nombre par ligne. Si l'option `-c` est utilisée, alors le script devra afficher les nombres `debut`, `debut+t`, `debut+2*t`, ..., sans jamais dépasser `fin`.

Exercice 3

Exécutez la commande suivante dans un terminal :

```
curl http://cboin.hd.free.fr/info1/fichiers-TD.tgz | tar xvpzf - -C /tmp
```

puis placez-vous dans `/tmp/fichiers-TD`. Ce répertoire contient des fichiers d'images qui sont toutes au format JPEG. L'extension des noms de fichiers est parfois correcte (`.jpg`), mais il arrive aussi qu'elle soit fausse (`.tif`, `.fig`, `.Z`, ...), ou inexistante.

Écrire un script qui corrige ce problème dans le cas général. Il recevra en unique paramètre un

répertoire, censé contenir les fichiers à corriger (/tmp/fichiers-TD dans le cas présent). Puis, pour chaque fichier de ce répertoire, écrira la ligne de commande nécessaire (sans la lancer) pour :

- renommer le fichier en .jpg si l'extension n'est pas la bonne
- l'ajouter s'il n'a pas d'extension

S'il n'a rien à faire car l'extension est la bonne, il l'écrira aussi. Aide : pour comparer deux chaînes de caractères entre elles, on peut utiliser la commande test avec l'opérateur =. Et le ! marque la négation (dans la commande test elle-même comme dans une condition en Bash). Comparez, par exemple, les codes de sortie de

```
test abc = abc
test ! xyz = abc
```

Exemple d'utilisation :

```
$ ./renommer.sh /tmp/fichiers-TD2
Je traite /tmp/fichiers-TD2/crabe 072.png -> mv
/tmp/fichiers-TD2/crabe 072.png /tmp/fichiers-TD2/crabe 072.jpg
Je traite /tmp/fichiers-TD2/image_0030.tif -> mv
/tmp/fichiers-TD2/image_0030.tif /tmp/fichiers-TD2/image_0030.jpg
Je traite /tmp/fichiers-TD2/image_0031.JFIF -> mv
/tmp/fichiers-TD2/image_0031.JFIF /tmp/fichiers-TD2/image_0031.jpg
Je traite /tmp/fichiers-TD2/image_0032.jpg -> est OK, rien à lancer
Je traite /tmp/fichiers-TD2/image_0033.jpg -> est OK, rien à lancer
Je traite /tmp/fichiers-TD2/image_0067.jpg -> est OK, rien à lancer
Je traite /tmp/fichiers-TD2/image_0071.jpg -> est OK, rien à lancer
Je traite /tmp/fichiers-TD2/inconnu.jpg -> est OK, rien à lancer
Je traite /tmp/fichiers-TD2/insecte .bmp -> mv
/tmp/fichiers-TD2/insecte .bmp /tmp/fichiers-TD2/insecte .jpg
Je traite /tmp/fichiers-TD2/Libell -> mv /tmp/fichiers-TD2/Libell
/tmp/fichiers-TD2/Libell.jpg
Je traite /tmp/fichiers-TD2/Une Libellile 010a.Z -> mv
/tmp/fichiers-TD2/Une Libellile 010a.Z /tmp/fichiers-TD2/Une Libellile
010a.jpg
$
```

Exercice 4

La commande `test -d dir` permet de vérifier si `dir` est un répertoire. Ecrire un script shell qui fera les choses suivantes pour chacun des arguments qu'il recevra :

- Il commencera par vérifier si cet argument est un répertoire. Si ce n'est pas le cas, il affichera « ... n'est pas un répertoire, ignoré. » sur sa sortie d'erreur standard.

- Sinon, deux cas de figure possibles :

- Soit le répertoire n'est pas vide : le script affichera alors « Le répertoire .. n'est pas vide, ignoré. » sur sa sortie d'erreur standard
- Soit il l'est : le script affichera alors « Le répertoire ... est vide, voulez-vous le supprimer (o/n) ? » sur sa sortie standard. Si l'utilisateur répond O ou o, alors il supprime le répertoire en question ; sinon, il ne fait rien.

Le script devra sortir avec le code de retour 0 si tous les arguments passés étaient bien des répertoires, sinon avec le code 1 (commande `exit`).

Exemple d'utilisation :

```

$ mkdir /tmp/vidé
$ ./répertoires.sh /etc/passwd /etc/ /tmp/vidé
/etc/passwd n'est pas un répertoire, ignoré.
Le répertoire /etc/ n'est pas vidé, ignoré
Le répertoire /tmp/vidé est vidé, voulez-vous le supprimer
(o/n) ?o
rmdir /tmp/vidé
$ echo $?
1

```

Exercice 5

Ecrire un script shell qui recevra un nombre quelconque d'arguments, censés être tous des répertoires. Puis, pour chacun de ces répertoires, fera en parallèle les choses suivantes :

Il lancera une commande `find -type d` sur ce répertoire.

La sortie standard de ce `find` sera redirigée vers le fichier `/tmp/find-xx.txt`, et la sortie d'erreur standard vers `/tmp/find-xx.log` où `xx` est un entier (pas forcément à 2 chiffres) correspondant au numéro de l'argument correspondant.

Le script devra sortir avec un code de retour égal au nombre de commandes `find` qui n'ont pas elles-mêmes terminé avec un code de retour nul.

Exemple de sortie :

```

$ ./parafind.sh /etc /var /tmp/vidé
J'attends le PID 14784
J'attends le PID 14785
J'attends le PID 14786
$ echo $?
2
$ ls -l /tmp/find-*
-rw-rw-r-- 1 xavier xavier 1028 18 mai 19:56 /tmp/find-1.log
-rw-rw-r-- 1 xavier xavier 90715 18 mai 19:56 /tmp/find-1.txt
-rw-rw-r-- 1 xavier xavier 9861 18 mai 19:56 /tmp/find-2.log
-rw-rw-r-- 1 xavier xavier 147288 18 mai 19:56 /tmp/find-2.txt
-rw-rw-r-- 1 xavier xavier 0 18 mai 19:56 /tmp/find-3.log
-rw-rw-r-- 1 xavier xavier 10 18 mai 19:56 /tmp/find-3.txt

```